Tutorial 1

Monday, 8 September 2025 23:28

SEARCH ALGORITHMS

Uninformed
blind search algos
- start node
- goal node
- frontier

BFS, DFS, UCS
DLS, IDDFS

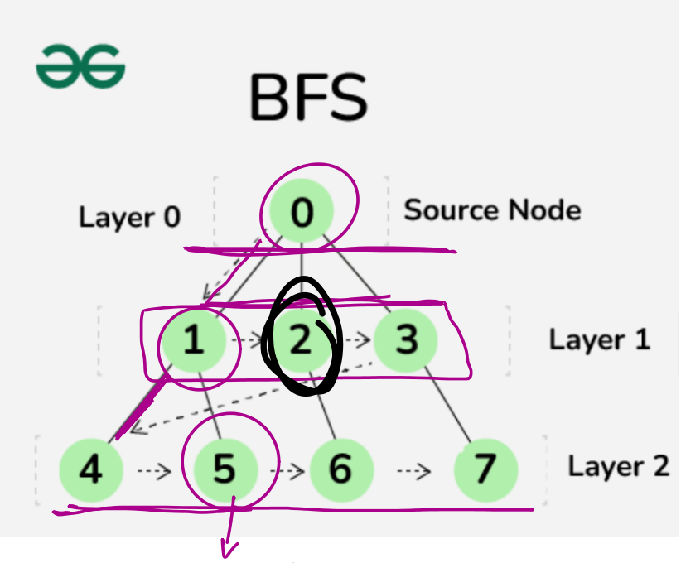
Informed
heuristic search

Greedy Search, A*

START NODE	entry point
GOAL NODE	target state
EXPLORED NODES	already visited and processed
UNEXPLORED NODES	not yet visited
FRONTIER NODES	potential node to be explored
TRAVERSAL	passing over

BREADTH - FIRST SEARCH (BFS)

- nodes at a given level/depth explored first before moving to next
- frontier is a queue — data structure approach
  FIFO - first in first out



start node = 0
goal node = 5

Queue : selected and removed

Step 1
At start node: 0
explored = { 0 }
frontier = 1, 2, 3

Step 2
At node: 1
explored = { 0, 1 }
frontier = 2, 3, 4, 5

Step 3
At node : 2
explored = {0, 1, 2}
frontier = 3, 4, 5, 6

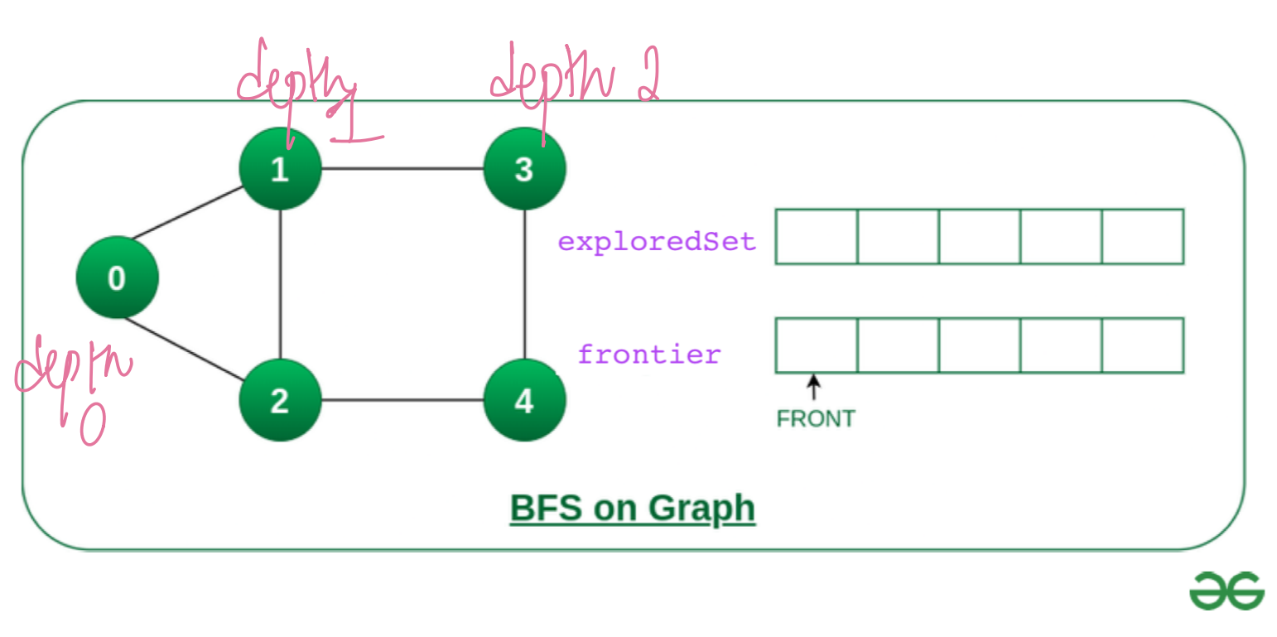
Step 4
At node : 3
explored = {0, 1, 2, 3}
frontier = 4, 5, 6, 7

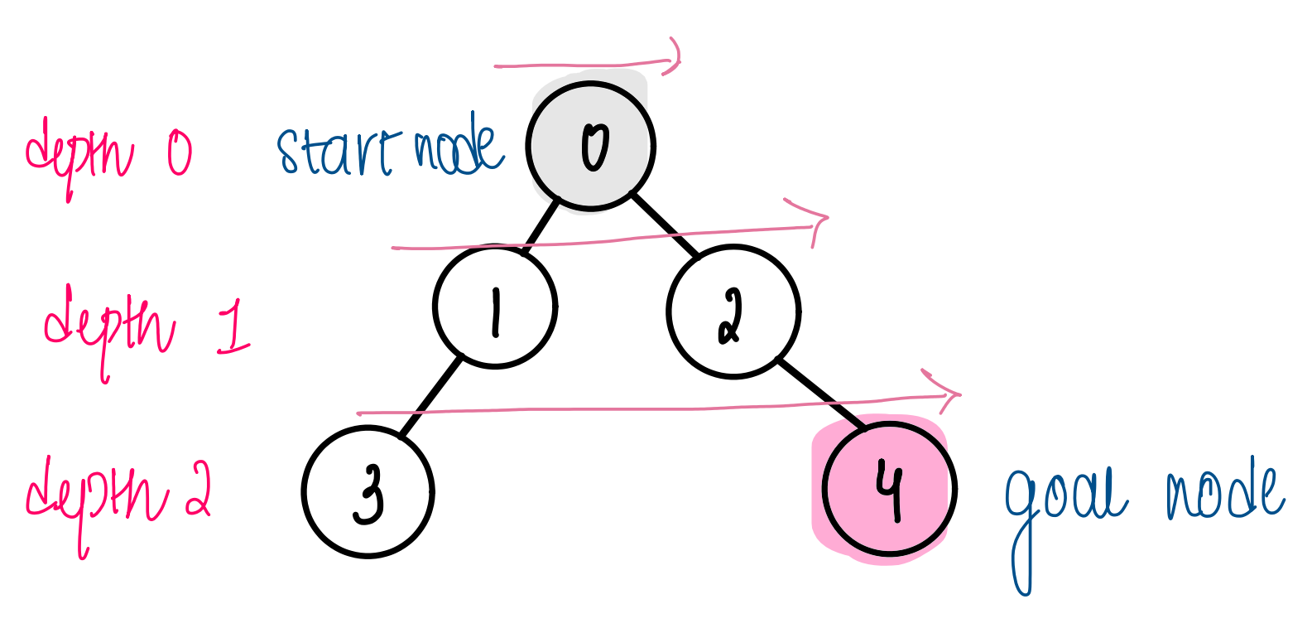
Step 4
At node : 4
explored = {0, 1, 2, 3, 4}
frontier = 5, 6, 7

Step 5
At node : 5
explored = {0, 1, 2, 3, 4, 5}
frontier = 6, 7 not relevant

Goal state acheived

Traversal = 0 → 1 → 2 → 3 → 4 → 5





Step 1
explored: {0}
frontier: 1, 2

Step 2
explored: {0, 1}
frontier: 2, 3

Step 3
explored: {0, 1, 2}
frontier: 3, 4

Step 4
explored: {0, 1, 2, 3}
frontier: 4

Step 5
explored: {0, 1, 2, 3, 4}
frontier: ø

0
2, 1

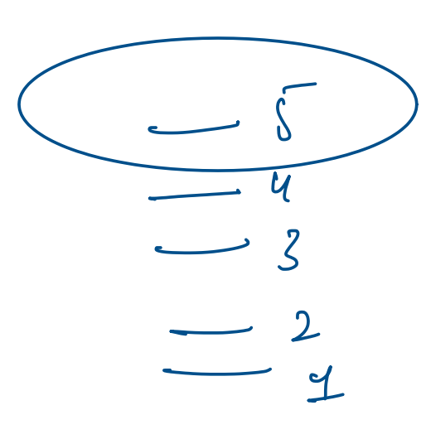
0, 2
1, 4

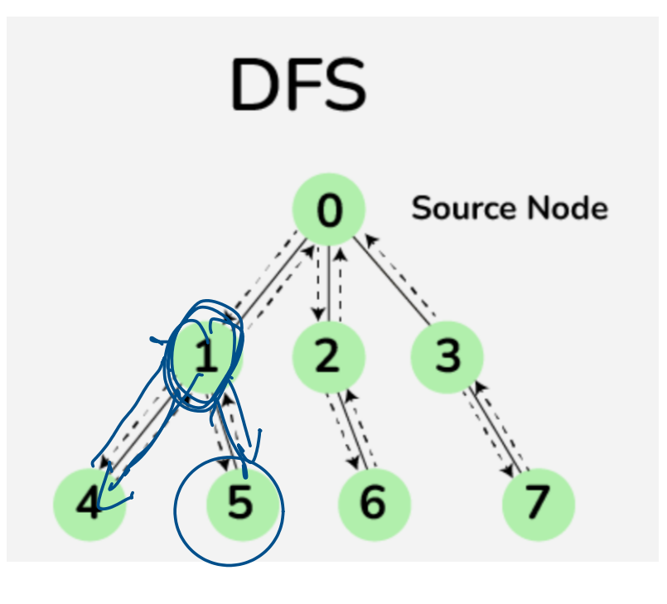
0, 2, 1
4, 3

0, 2, 1, 4
3

DEPTH-FIRST SEARCH (DFS)

- traversal begins at start node and goes down the layers until there is a node with no unexplored neighbours
- stack — LIFO
  Last-in first out





start node = 0
goal node = 5

Step 1
At start node: 0
explored = { 0 }
frontier = 1, 2, 3

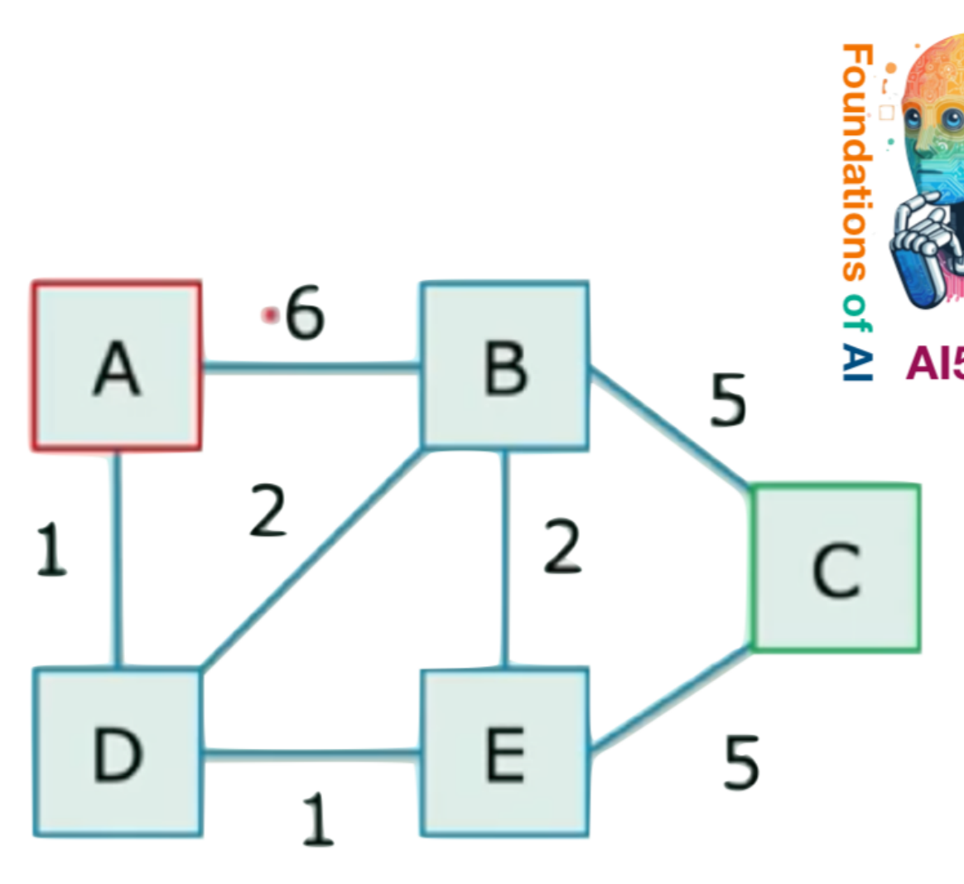
Step 2
At node: 1
explored = { 0, 1 }
frontier = 2, 3, 4, 5

Step 3
At node : 4
explored = {0, 1, 4}
frontier = 3, 2, 5

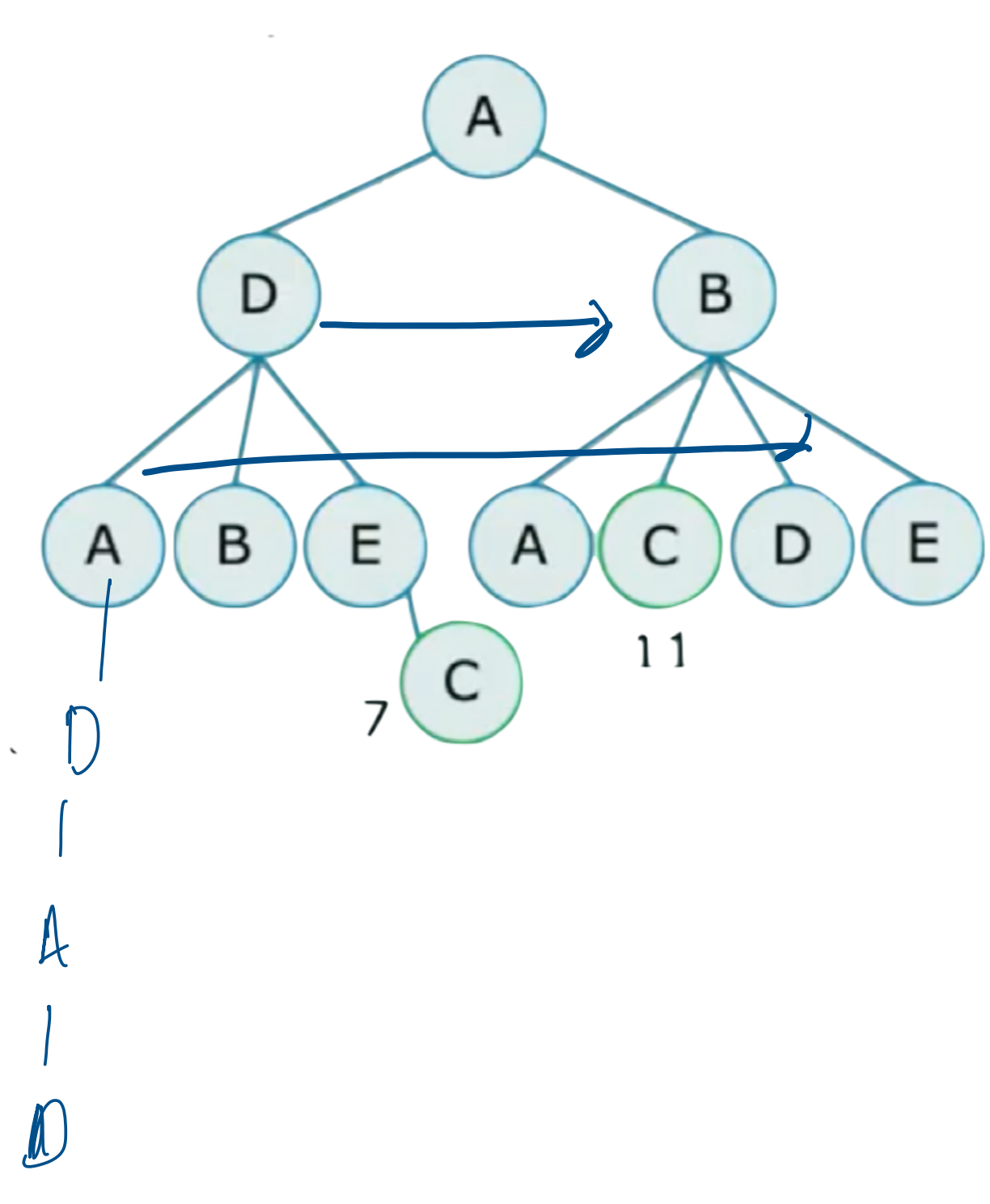
Step 4
At node : 5
explored = {0, 1, 4, 5}
frontier = 3, 2 not relevant

Goal state acheived

Traversal = 0 → 1 → 4 → 5



BFS	DFS
complete	not complete
not optimal	not optimal
Time complexity: $O(b^d)$	Time complexity $O(b^m)$
Space complexity: $O(b^d)$	Space complexity $O(bm)$



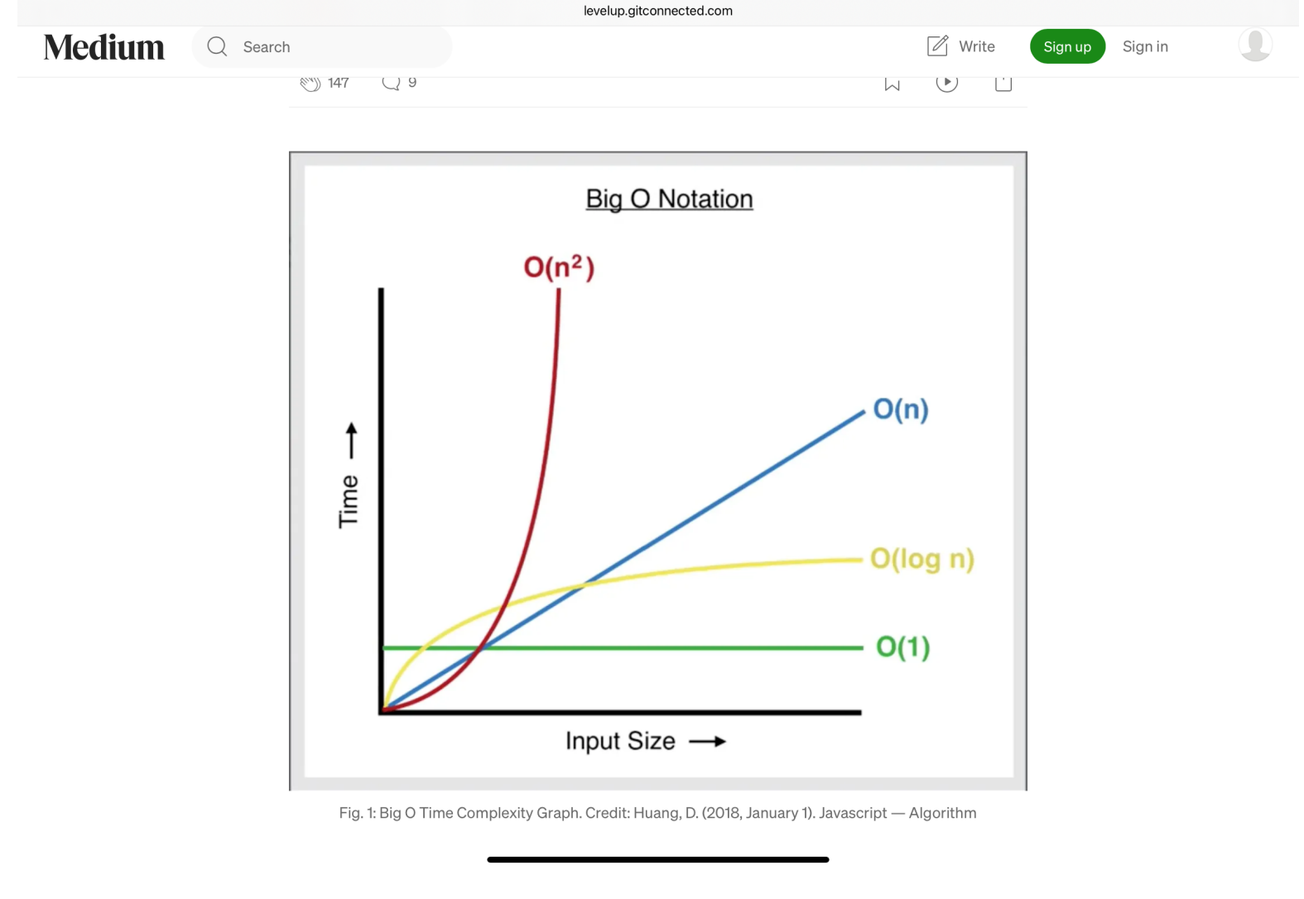
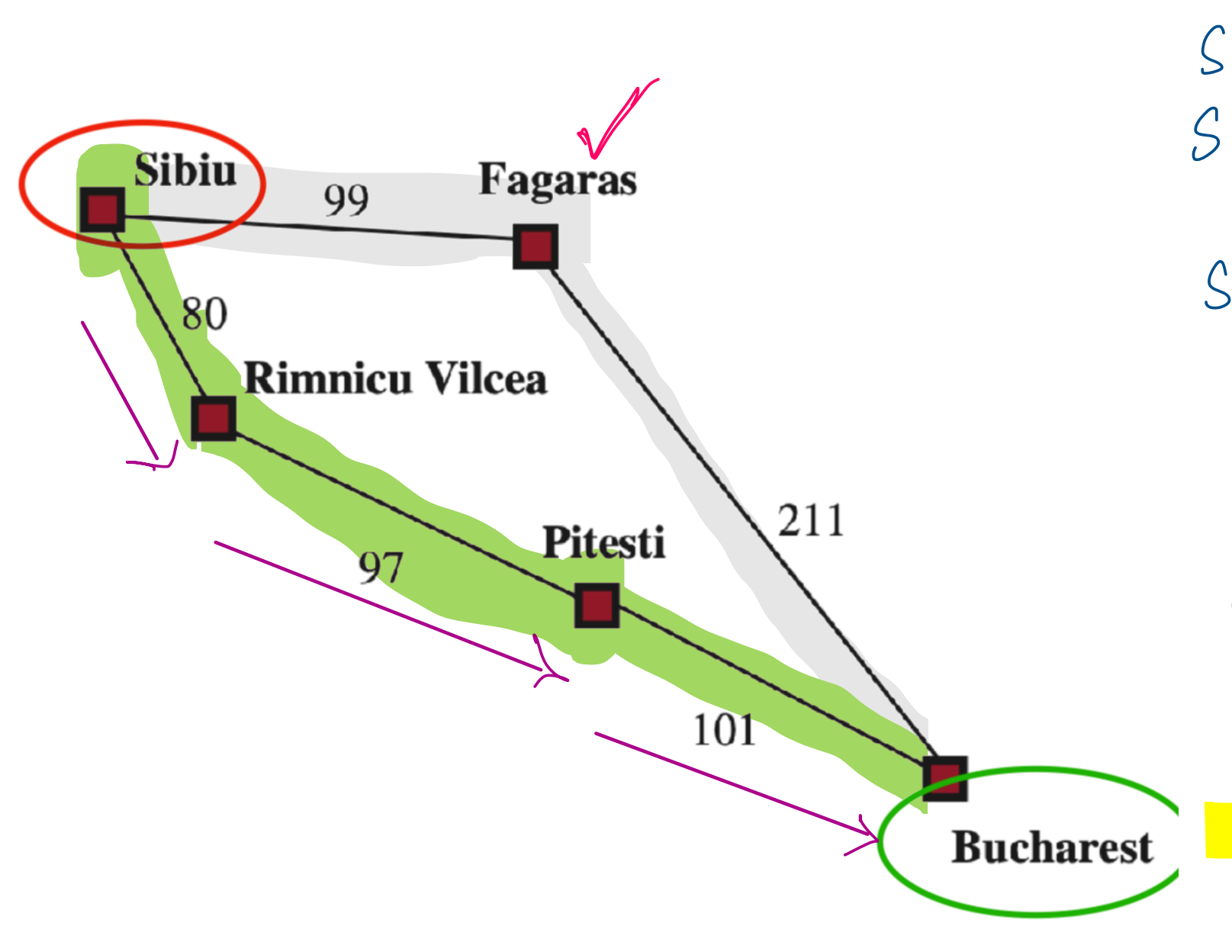


Fig. 1: Big O Time Complexity Graph. Credit: Huang, D. (2018, January 1). Javascript — Algorithm

UNIFORM COST SEARCH (UCS)

- least cost path
- Dijkstra Algorithm

uses queue - based on cost estimation
priority queue



S to F = 99
S to RV = 80 further explore

S to RV to P = 80+97 = 177
177 is more than S to F cost so that is now explored

S to F to B = 99 + 211 = 310
310 > 177 so that is explored

S to RV to P to B = 80 + 97 + 101 = 278
least cost path

- complete
- optimal
- complexity:

$O(b^{1+C^*/\epsilon})$

C^* path cost to goal
ϵ step count (min)